SOFTWARE DEV FRAMEWORK

Autopilot SDLC

From Idea to Delivery with AI Copilots

A framework for autonomous software development using Codex CLI, context engineering, and phase-gated execution.

Anurag Kumar

THE PROBLEM

AI can write code. But shipping software is not writing code.

Context Collapse

LLMs lose track of requirements mid-build. No memory between sessions.

Scope Explosion

Without constraints, AI agents gold-plate features nobody asked for.

No Checkpoints

No human review points means errors compound silently across phases.

Zero Documentation

Code appears fast, but nobody knows why decisions were made.

THE SOLUTION

Autopilot SDLC

A cognitive architecture that encodes the entire Agile SDLC into a single framework — turning Codex CLI into a disciplined engineering team, not a code generator.

Context Engineering

Layered knowledge injection. Only load what's needed, when it's needed.

Human Copilot Protocol

Autonomous between checkpoints. Collaborative at decision gates.

Documentation Driven

Docs before code. Living specs, journals, and wiki as source of truth.

THE HUMAN COPILOT MODEL

You set the destination. The agent flies the route.

YOU (Team Principal)

- Provide the product idea
- Approve vision & architecture
- Review at sprint checkpoints
- Redirect when needed
- Accept final delivery

CODEX (Copilot Agent)

- Decomposes idea (WHY → HOW → WHAT)
- Designs architecture & sprint plan
- Builds, tests, documents each sprint
- Writes dev journals & maintains wiki
- Commits, tags, and delivers

CONTEXT ENGINEERING

Layered Knowledge Injection

Only ~130 lines in context at any time, instead of 500+. Each layer loads on demand.

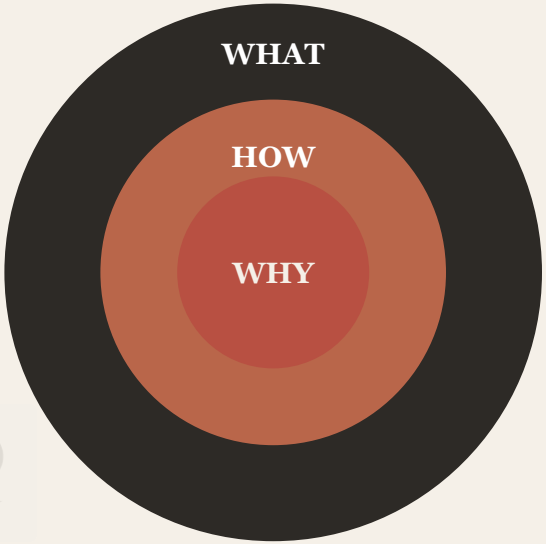
Layer 1	**Global AGENTS.md**	~30 lines. Engineering identity, universal standards. Every project.
Layer 2	**Project AGENTS.md**	~100 lines. Current state, project map, operating principles. Bootloader.
Layer 3	**Phase Runbooks**	~80-120 lines each. Detailed playbooks. Only current phase loaded.
Layer 4	**Docs / Wiki / Journal**	Living project knowledge. Read as needed for deep context.

The "bootloader" pattern: AGENTS.md is a program counter, not an instruction manual. It points to the right runbook.

PHASE 1: DISCOVERY

The Golden Circle

Every project starts with WHY — before a single line of code is written.



WHY

What problem exists? Who suffers?
What's the deeper motivation?

HOW

Strategic approach, constraints,
anti-scope (what you won't build)

WHAT

MVP features, user journey,
measurable success criteria

THE PIPELINE

Six Phases, Five Gates

0	1	2	3	4	5
Intake	**Discovery**	**Architecture**	**Construction**	**Hardening**	**Delivery**
Idea provided	Vision approved	Plan approved	Sprint approved	Quality approved	Handoff complete

Every gate requires explicit user approval. The agent cannot skip ahead.

PHASE 3: CONSTRUCTION

The Sprint Cycle

1 Plan

Read tasks, identify order

2 Build

Test-first, implement, verify

3 Journal

Post-build dev journal entry

4 Commit

feat(sprint-N): description

5 Test

Full suite, fix failures

6 Journal

Post-test retrospective

7 Tag

v0.N.0 + changelog update

8 Wiki

Update codebase map & patterns

CHECKPOINT → Present sprint results to user → Wait for approval → Next sprint or adjust

LIVING KNOWLEDGE

Three Knowledge Systems

Documentation

- WHY (VISION.md)
- Contracts (REQUIREMENTS.md)
- Blueprint (ARCHITECTURE.md)
- Backlog (TASKS.md)
- ADR log (DECISIONS.md)

Dev Journals

- What Happened
- What Broke & Why
- Key Insights
- Hardening
- Flight Recorder
- Debugging

Project Wiki

- Project Overview (HOME.md)
- CODEBASE-MAP.md
- Standards & Conventions (PATTERNS.md)
- Debug Playbook (TROUBLESHOOTING.md)
- Versioned Snapshots

METHODOLOGIES ENCODED

Seven Disciplines, One Framework

Golden Circle Framework	WHY → HOW → WHAT decomposition before any technical work	Phase 1
Knowledge Engineering	Layered context injection - load only what's needed, when it's needed	All Phases
Documentation-Driven Dev	Write the spec before writing the code. Docs are the source of truth.	All Phases
Spec-Driven Development	Requirements as contracts. Every feature traces to a requirement.	Phase 2-3
Intent-Driven Development	Everything traces back to the WHY. Anti-scope is scope.	All Phases
Systems Thinking	Boundaries, interfaces, feedback loops. Second-order effects considered.	Phase 2-4
Human Copilot Protocol	Autonomous between checkpoints. Collaborative at decision gates.	All Phases

FOR LEADERSHIP

What Changes for Your Team

Idea → Working MVP	Weeks to months	→	**Hours to days**
Traceability	Scattered or absent	→	**Full audit trail: docs, journal, wiki**
Human Oversight	Ad-hoc code review	→	**Structured checkpoints at every gate**
Knowledge Retention	Lost in Slack threads	→	**Living wiki with versioned snapshots**

GETTING STARTED

Three Steps to Launch

01 Install the Global DNA

Copy GLOBAL-AGENTS.md to ~/.codex/AGENTS.md.
Customize with your team's coding standards, git conventions, and quality expectations.

02 Initialize a Project

Copy the project AGENTS.md and runbooks to your repo root.
Run git init. Commit the framework.
The structure is your operating system.

03 Launch Codex & Describe Your Idea

Run codex in your terminal. The agent reads the AGENTS.md, detects Phase 0, and prompts you for your product idea.
The workflow takes over.

AUTOPILOT SDLC

Don't just generate code. Engineer software.

Anurag Kumar
Founder, Prex Studio
Available on **GitHub**